Faith User

Academic Writting

Thesis
Research
University of Technology Sarawak

Document Details

Submission ID
trn:oid:::1:3460320851

Submission Date
Nov 2, 2026, 7:15PM GMT+8

Download Date
Nov 2, 2026, 7:22 PM GMT+8

File Name
Academic Writting.docx

File Size
15.4 KB

7 Pages

2,204 Words

15,209 Characters

43% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

88 AI-generated only 43%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Generative AI in Code Generation and Debugging: A Literature Review on Software Engineering Learning Among Data Engineering Students

Abstract

Tools that use Generative AI (GenAI), such as GitHub Copilot and ChatGPT, are increasingly being used in software engineering education for code writing and debugging tasks. However, the impact of these tools on learning among data engineering students is not clear, and the number of studies dedicated to this subgroup is limited. The present work is a literature review of existing studies about the impact of GenAI on students' learning in data engineering and related computing fields. The relevant literature was found through academic databases using keywords related to GenAI, code writing, debugging and software engineering education. The results reveal that GenAI makes it possible to receive quick feedback and explanations provided about coding tasks, but they also raise important concerns about the reliance on such technology, incorrect information provided, and a cloud of authorship around the material produced. It is recommended that students should always double check the results generated using AI technologies, and make sure that they depend on their own thinking rather than allowing artificial intelligence (AI) to make decisions for them. Educators need to focus on the idea of AI literacy and consider its teaching in their classes.

Introduction and Problem Statement

Applications of Generative Artificial Intelligence (GenAI) such as ChatGPT and GitHub Copilot are becoming popular in computer programming education for the purpose of code writing, error correction or clarification (IJACSA authors, 2026; Strik, Menolli and Brancher, 2024). For instance, students enrolled in software engineering programs can describe a certain task and get a working code almost instantly or simply write something wrong and obtain a fix right away. This is very important for the education of data engineers, as high quality of pipelines hinges on development and implementation and life cycle management techniques as much as on data processing skills (Foidl et al., 2023). However, acquiring these techniques requires plenty of effort that makes the temptation to get quick AI answers even higher among students.

Previous research has indicated that GenAI can help improve effectiveness and aid creativity. However, there are certain issues raised about the occurrence of erroneous output, propagation of

mistakes and unclear authorship (Reihanian et al., 2024). For example, while debugging, beginner programmers often place excessive dependence upon the model,simply following its educational advice (Bo et al., 2025); furthermore, solutions that give total fixes may adversely affect independent debugging capabilities (Kurniawan et al., 2025). In addition, most evidence comes from introductory programming courses and computing, while research of the data-focused users is just beginning (Bien and Mukherjee, 2025).

Thus, the difficulty lies in the fact that there is no properly substantiated evidence of how GenAI helps or disrupts software engineering learning in the context of code generation and debugging among data engineering students, which complicates providing any recommendations about its usage. The paper aims to fill the existing void in the field by reviewing the current literature on how GenAI assists or disrupts the development of software engineering knowledge in terms of code generation and debugging for data engineering learners.

Research Objectives and Questions

Research Objective (RO)

- RO1: Identify how GenAI tools are utilized for creating programs and fixing bugs in software engineering education among data engineering and related information technology students.
- RO2: Examine advantages and disadvantages of these applications mentioned in research publications.
- RO3: Propose ways for learners and educators to apply GenAI safely in the field of software engineering education.

Research Questions (RQ)

- RQ1: What are the methods for applying GenAI in coding and debugging in students from fields concerning data engineering and computation?
- RQ2: What are the possible advantages and disadvantages of the mentioned approaches according to studies?
- RQ3: Which practices allow the use of GenAI in an efficient and safe way within the realm of software engineering learning?

Literature Review

The rapid evolution of Generative Artificial Intelligence (GenAI), including large language models (LLMs) and intelligent coding assistants (ICAs) such as GitHub, Copilot, and ChatGPT, has fundamentally reshaped software engineering (SE) education and practice. GenAI brings both important learning opportunities and unique educational challenges to data engineering students who are typically juggling complex data pipeline scripts, database queries and algorithmic logic. Recent works address these impacts along three main axes: developer productivity and code generation benefits, debugging support and interaction dynamics, and security risks of academic overreliance.

1. **Code Generation and Productivity Enhancement**
   One of the main drivers for the adoption of GenAI in technical education is its capacity to enhance programming productivity and simplify the software development lifecycle (SDLC). Studies have shown that AI code assistants can speed up routine coding tasks, help students stitch template code together, and fill in knowledge gaps when students encounter unfamiliar data structures or APIs. GenAI is able to act as an on-demand mentor in data engineering education where students often spend a lot of time writing repetitive configuration scripts and data transformation logic, minimizing syntactic friction and keeping them in motion.

2. **Interaction Dynamics during Debugging and Exploration**
   It is important to go beyond just writing code and to understand how students engage with AI as they debug and troubleshoot. Barke et al. (2023) present groundbreaking empirical research showing that developer-AI interactions are predicated on bimodal behaviors which are an acceleration mode where AI is leveraged to quickly perform well understood logic and an exploration mode where deep prompting, multi-suggestion comparisons, and line-by-line validation are used to resolve unfamiliar errors. Debugging AI generated code requires active critical thinking rather than passive acceptance especially when dealing with complex runtime errors or logic bugs in data pipelines.

3. **Security Risks, Code Errors, and Overreliance**
   Despite the clear utility, recent literature sounds important cautionary notes regarding the risks of unrestrained GenAI usage. Comprehensive security analyzes show that LLM generated code can sometimes introduce subtle security vulnerabilities, syntax flaws or hidden logic errors. Additionally, educational researchers warn of the risk of overreliance especially among beginner programmers who may overestimate AI reliability. Simply taking AI output and using it without thinking can lead to subtle plagiarism, erosion of basic problem-solving skills, and a shortcutting of the deep cognitive processing necessary to truly master software engineering principles.

Research Methodology

To investigate the role of Generative Artificial Intelligence (GenAI) in code generation and debugging among data engineering students, this study adopted a systematic literature review (SLR) approach. The methodology outlines the systematic process of gathering, screening, and evaluating relevant peer reviewed literature to synthesize current insights.

1. **Review Process and Approach**
    - Identification: Formulating clear search strings and identifying appropriate digital repositories.
    - Screening: Filtering retrieved papers based on predefined inclusion and exclusion criteria that focus on the title, abstract, and relevance to software or data engineering education.
    - Selection: Reviewing full articles to ensure they directly address code generation, debugging behavior, productivity or risks associated with AI assistants.

2. **Digital Databases Used**
    - Google Scholar: For broad multidisciplinary coverage of open access and preprint scholarly articles.
    - IEEE Xplore: To capture peer-reviewed conference papers and journals focusing specifically on software engineering, computing education, and artificial intelligence.

3. Inclusion and Exclusion Criteria
    - Inclusion Criteria
        - Peer-reviewed journal articles, conference proceedings, and empirical studies.
        - Literature published within the recent timeframe to reflect rapid advancements of modern GenAI models.
        - Studies focusing on programming education, developer workflows, interaction modes or software engineering learning outcomes.
    - Exclusion Criteria
        - Non-academic sources such as blog posts, tutorials, and unverified news articles.
        - Papers lacking full text accessibility or written in language other than English.
        - Studies focusing purely on hardware engineering or non-programming domains without direct relevance to software or data engineering.

Results and Discussion

The Systematic Literature Review (SLR) methodology was used to synthesise the empirical findings. The SLR methodology was described in the research methodology section where it employs online repositories such as Google Scholar, IEEE Xplore, ACM Digital Library, Springer, and MDPI. The findings respond to Research Question 1 and Question 2 about the effects of using Generative Intelligence (GenAI) tools on code generation and debugging for data engineering and computing students. The example tools used are Github Copilot and ChatGPT.

Table 1 below provides a clear synthesis of current research. The table compares the primary studies examined, summarising their core focus, key findings on coding and debugging, and identified learning risks.

Table 1: Summary and Comparison of Selected Literature

Study Reference	Primary Focus	Key Findings on Code Generation & Debugging	Pedagogical Risks
Barke et al. (2023)	Empirical study on Copilot.	Identifies bimodal behaviours: Acceleration Mode & Exploration Mode.	Accepting the generated code without question can lead to logic bugs.
Yalwa et al. (2026)	Systematic Review of GenAI for Programming in Higher Education.	Reduce syntactic friction by auto-generating code snippets, routine .	Code hallucinations, unverified outputs, and skill weakening.
Strik et al. (2024)	Systematic mapping of GPT AI tools in computer science education.	Instant feedback, explaining error tracebacks and translating complex compiler messages.	Passive use of AI tools reveals a lack of pedagogical frameworks.
Foil et al. (2023)	Data pipeline quality analysis.	Importance of rigorous script development for pipeline infrastructure.	Complex data dependencies can lead to silent bugs.
Bo et al. (2025)	AI debugging assistants for beginner programmers.	Speed up mistakes and instant code fixes.	High level of passive acceptance and overreliance on AI.
Kurniawan et al. (2025)	AI assisted debugging pilot study.	Assists learners in navigating complex runtime exceptions.	Instant automated fixes impair independent

			debugging skills.
Reihanian et al. (2024)	Scoping review of AI risks,	Generates code templates faster for all computer science tasks.	Code hallucinations, unoptimized queries, and security risks.
Bien & Mukherjee (2025)	Data engineering education and software development life cycles.	Simplifies learning across data processing pipelines.	Limited empirical focus on data-specific learners.

GenAI is mainly used for code generation tasks by students. For example, SQL boilerplate and setup scripts and error interpretation tasks (Yalwa et al., 2026; Strik et al., 2024). Barke et al. (2023) explained that when students use GenAI, they switch between Acceleration Mode and Exploration Mode. Acceleration Mode helps them to write basic code or routine code, while Exploration Mode helps them to fix errors or solve complex bugs. There are several important benefits for students when they use GenAI, which is that it gives them instant support and removes the frustration of memorising exact syntax (Strik et al., 2024; Bien & Mukherjee, 2025). However, GenAI can also make up fake code, create weak code that can lead to security vulnerabilities (Reihanian et al., 2024), make students passive (Bo et al., 2025), and stop students from learning how to debug code on their own. In addition, data engineering is about building big systems such as how data flows through pipelines and databases (Foidl et al., 2023; Bien & Mukherjee, 2025). Although GenAI is great at writing small and quick code snippets, when it needs to correct complicated data systems, GenAI often provides wrong or broken advice. In conventional software engineering, runtime errors typically detect immediate execution failures that alert the developer. In contrast, flawed AI-generated scripts in data engineering often execute successfully. It does not throw explicit compiler errors and silently corrupts data warehouses (Foidl et al., 2023). Thus, blindly trusting GenAI (Bo et al., 2025; Kurniawan et al., 2025) endangers the integrity of data.

Conclusion and Recommendations

Based on the synthesised findings, clear patterns of how students use GenAI, key technical trade-offs, and action plans that universities and lecturers need to consider to handle AI safely were revealed. Students mostly ask AI to help them write for routine boilerplate, format SQL queries, and explain error messages (Yalwa et al., 2026; Strik et al., 2024). When using AI, students switch between two modes, which are Acceleration Mode and Exploration Mode (Barker et al., 2023). By using GenAI, students spend less time facing syntax rules, and they can get fast feedback (Strik et al., 2024). On the other hand, the drawbacks of using GenAI among students can lead to the invention of code hallucinations (Reihanian et al., 2024), make students become lazy (Bo et al., 2025), and introduce hidden errors in database pipelines (Foidl et al., 2023). Hence, to ensure safe and effective AI usage, students must actively check AI outputs, and universities must formally teach AI literacy among students in data engineering. Several practical recommendations are proposed for learners and educators to apply GenAI safely in the field of software engineering education. Students should read

the SQL or ETL code generated by AI line-by-line (Reihanian et al., 2024) to ensure it will not silently ruin any data. Additionally, students should try fixing errors independently first before requesting GenAI for the answer (Barke et al., 2023; Kurniawan et al., 2025). Moreover, universities should teach students AI literacy so students know how to spot AI mistakes and assess code risks. In addition, students should be graded on oral code defences, designing structures, and doing live code review. Finally, explicit institutional guidelines should be set up defining what levels of AI assistance are acceptable. In conclusion, GenAI helps students finish code quickly and efficiently. Data engineering students must balance AI productivity with critical thinking and manual verification, and have real technical understanding to ensure they acquire strong, self-sufficient software engineering skills.